

TEAM A — Notification Service · API Gateway · Platform & Integration

Owner: Arham Ibrahim Khan (220042160) Your services: **Notification Service (8007)** and **API Gateway (8000)**. You also own the platform (**Docker, replication, broker**) and the final merge.

Give this whole file to Claude Code in the repo root and say: "Read TEAM_A_INTEGRATION.md and implement my sections completely. *libs/common/* and *frontend/* (shell) already exist — read them, import from them, do not rewrite them."

SECTION 1 — THE SHARED CONTRACT

(Sections 1.1–1.13 are byte-identical in all four team documents. Never change them unilaterally — message the group first.)

1.1 Who builds what (roughly equal load, everyone owns real services)

	Person	Services owned	Also owns	Est. hours
A	Arham Ibrahim Khan (220042160)	Notification (8007), API Gateway (8000)	Docker Compose + all infra + Postgres replication, <i>/notifications</i> UI, docs & diagrams, final merge	~15.5
B	Arham Apon Utsho (220042153)	User (8001), Account (8002)	login/register/wallet/dashboard UI, <i>scripts/smoke_test.py</i> , market-maker bot	~13.5
C	Abidur Rahman Asif (220042152)	Order (8003), Matching Engine (8004)	orders UI + OrderTicket/OrderBook/RecentTrades components	~13.5
D	Mustain Billah Taj (220042166)	Market Data (8005), Portfolio (8006)	market page + candlestick chart + portfolio UI, candle seeder, mock API	~15

All 7 proposal services + the Gateway are covered. Two services each.

1.2 Ownership map — nobody edits anybody else's files

Path	Owner
<i>libs/common/**</i>	pre-built, frozen (A is the

Path	Owner
	only one who may patch it)
frontend/lib/**, frontend/app/layout.tsx, frontend/app/globals.css, frontend/components/{ui, Toast, Protected, Navbar}.tsx, frontend/package.json, configs	pre-built, frozen
requirements-base.txt, .gitattributes, .dockerignore	pre-built
docker-compose.yml, .env, Makefile, infra/postgres/**, docs/**, README.md	A
services/gateway/**, services/notification-service/**	A
frontend/app/notifications/page.tsx, frontend/components/NotificationBell.tsx	A
services/user-service/**, services/account-service/**	B
frontend/app/{login, register, wallet}/page.tsx, frontend/app/page.tsx, frontend/components/{SymbolTable, DepositForm}.tsx	B
scripts/smoke_test.py, infra/seed/market_maker.py	B
services/order-service/**, services/matching-engine/**	C
frontend/app/orders/page.tsx, frontend/components/{OrderTicket, OrderBook, RecentTrades, OrdersTable}.tsx	C
services/market-data-service/**, services/portfolio-service/**	D
frontend/app/market/[symbol]/page.tsx, frontend/app/portfolio/page.tsx, frontend/components/{CandleChart, HoldingsTable}.tsx	D
infra/seed/seed_market_data.py, frontend/mock-api.py	D

1.3 Ports

Component	Container	Port	Host
Frontend	frontend	3000	3000
API Gateway	gateway	8000	8000
User	user-service	8001	8001
Account	account-service	8002	8002
Order	order-service	8003	8003
Matching Engine	matching-engine	8004	8004
Market Data	market-data-service	8005	8005
Portfolio	portfolio-service	8006	8006

Component	Container	Port	Host
Notification	notification-service	8007	8007
Postgres user / account / order	postgres-user / -account / -order	5432	5433 / 5434 / 5435
Postgres market PRIMARY / REPLICA	postgres-market-primary / -replica	5432	5436 / 5437
Postgres portfolio / notification	postgres-portfolio / -notification	5432	5438 / 5439
Redis	redis	6379	6379
RabbitMQ	rabbitmq	5672 / 15672	5672 / 15672

Matching Engine has **no database** — the book is in memory, by design.

1.4 Money & quantity rules

- Money is `Decimal`, 4 dp, `ROUND_HALF_UP`. **Never float**. Postgres `NUMERIC(18,4)`.
- Money crosses the wire as a **JSON string**: `"195.5000"`. (One documented exception: `/api/market/candles` returns numbers, because `lightweight-charts` needs them.)
- Quantity is a positive **int** (whole shares). Postgres `BIGINT`.
- Limit price must be a positive multiple of `0.01`.
- Use `common.money: to_money, money_str, validate_price, validate_quantity, notional`.
- Symbols: `common.symbols.SYMBOLS, SEED_PRICES, normalize_symbol`.

1.5 Auth

JWT HS256, secret `JWT_SECRET` **identical in every container**. Claims: `sub` (user id UUID string), `email`, `username`, `iat`, `exp`, `type:"access"`. Expiry 1440 min. Clients send `Authorization: Bearer <token>`. The gateway validates **and forwards the header unchanged**; every service validates again via `common.security.get_current_user`. `/internal/*` requires `X-Internal-Key` (`Depends(require_internal_key)`) and **the gateway refuses to proxy any path containing `/internal/`**.

1.6 Gateway routing table — the frontend's entire world

Strip `/api`, keep the rest of the path and the query string.

Method	Public path	Upstream	Auth
POST	<code>/api/auth/register</code>	user-service	public
POST	<code>/api/auth/login</code> (form-encoded)	user-service	public
POST	<code>/api/auth/login-json</code>	user-service	public
GET/PATCH	<code>/api/users/me</code>	user-service	JWT
GET	<code>/api/account/balance</code>	account-service	JWT

Method	Public path	Upstream	Auth
POST	/api/account/deposit	account-service	JWT
GET	/api/account/transactions	account-service	JWT
POST	/api/orders	order-service	JWT, 30/min
GET	/api/orders, /api/orders/{id}	order-service	JWT
DELETE	/api/orders/{id}	order-service	JWT
GET	/api/book/{symbol}	matching-engine	public
GET	/api/market/symbols, /quote/{s}, /candles/{s}, /trades/{s}	market-data-service	public
GET	/api/portfolio, /api/portfolio/pnl	portfolio-service	JWT
GET	/api/notifications, /api/notifications/unread-count	notification-service	JWT
POST	/api/notifications/{id}/read, /api/notifications/read-all	notification-service	JWT
WS	/ws/market	market-data-service /ws/market	public
GET	/health	gateway itself	public

Errors everywhere are {"detail": "human readable message"} with a real status: 400 validation, 401 auth, 403 not yours, 404 missing, 409 business conflict, 429 rate limited, 503/504 upstream.

1.7 Event contract

Exchange `exchange.events` (topic, durable). Dead-letter `exchange.events.dead`. Envelope:

```
{ "event_id": "<uuid4>", "event_type": "trade.executed",
  "occurred_at": "2026-08-08T12:00:00.000000Z", "version": 1, "payload": { } }
```

Routing key	Published by	Consumed by
order.accepted	Order (C)	Matching Engine (C)
order.rejected	Order (C)	Account (B), Portfolio (D), Notification (A)
order.cancel_requested	Order (C)	Matching Engine (C)
order.cancelled	Matching Engine (C)	Order (C), Account (B), Portfolio (D), Notification (A)

Routing key	Published by	Consumed by
<code>order.cancel_rejected</code>	Matching Engine (C)	Order (C)
<code>trade.executed</code>	Matching Engine (C)	Order (C), Account (B), Portfolio (D), Market Data (D), Notification (A)

```
// order.accepted
{"order_id","user_id","symbol","side":"BUY|SELL","order_type":"LIMIT|MARKET",
 "price":"195.5000"|null,"quantity":10,"created_at":"...Z"}
// order.rejected
{"order_id","user_id","symbol","reason","rejected_at"}
// order.cancel_requested
{"order_id","user_id","symbol","side","requested_at"}
// order.cancelled
{"order_id","user_id","symbol","side","cancelled_quantity":7,
 "reason":"USER_REQUEST|IOC_REMAINDER|NO_LIQUIDITY","cancelled_at"}
// order.cancel_rejected
{"order_id","user_id","reason":"NOT_IN_BOOK|ALREADY_FILLED","rejected_at"}
// trade.executed
{"trade_id","symbol","price":"195.5000","quantity":5,
 "buy_order_id","sell_order_id","buyer_user_id","seller_user_id",
 "aggressor_side":"BUY|SELL","buy_order_remaining":0,"sell_order_remaining":3,
 "buy_order_limit_price":"196.0000"|null,"executed_at"}
```

Why the Matching Engine — not the Order Service — publishes `order.cancelled` (a deliberate refinement of the proposal; put this in the README): only the book knows whether the order was still resting when the cancel arrived. If the Order Service released funds optimistically, a fill landing in the same millisecond would settle against a reservation that no longer exists. Two-phase cancel (`cancel_requested` → engine removes → `cancelled`) makes the compensation race-free.

Queue names (each consumer declares its own; never share a queue):

```
q.matching.order_accepted      q.matching.cancel_requested
q.order.trade_executed         q.order.order_cancelled
q.order.cancel_rejected
q.account.trade_executed       q.account.order_cancelled
q.account.order_rejected
q.portfolio.trade_executed     q.portfolio.order_cancelled
q.portfolio.order_rejected
q.marketdata.trade_executed
q.notification.trade_executed  q.notification.order_cancelled
q.notification.order_rejected
```

1.8 Redis keys

```

lock:funds:{user_id}          distributed lock, SET NX PX 5000, Lua compare-and-
delete release
lock:shares:{user_id}:{SYMBOL} same
idem:{service}:{event_id}    SET NX EX 86400 – event idempotency
md:last_price:{SYMBOL}       "195.5000" (Market Data writes; Portfolio +
Order read)
md:quote:{SYMBOL}            {"price","quantity","ts"}
ratelimit:{identity}:{minute} INCR + EXPIRE 60 (gateway)
CHANNEL md:ticks              {"symbol","price","quantity","ts"}

```

Helpers: `common.redis_client.distributed_lock`, `already_processed`, `clear_processed`, `get_last_price`, `set_last_price`.

1.9 Internal service-to-service REST

```

POST account-service:8002 /internal/reservations
  {"order_id","user_id","amount":"1020.0000"}
  201 {"reservation_id","order_id","amount","status":"HELD"}
  409 {"detail":"insufficient buying power"} 503 lock timeout
  Idempotent on order_id.
POST account-service:8002 /internal/reservations/{order_id}/release -> 200
{"released":"120.0000"}

POST portfolio-service:8006 /internal/share-reservations
  {"order_id","user_id","symbol","quantity":10}
  201 {"reservation_id","order_id","quantity","status":"HELD"}
  409 {"detail":"insufficient shares"}
  Idempotent on order_id.
POST portfolio-service:8006 /internal/share-reservations/{order_id}/release -> 200
{"released":10}

GET order-service:8003 /internal/orders/open -> open orders, for engine book
rebuild
GET user-service:8001 /internal/users/{id} ->
{"id","email","username","full_name"}
GET matching-engine:8004 /book/{symbol}?depth=10 (public)

```

Use `common.http_client.call_service` — sets `X-Internal-Key`, retries transport errors, raises `ServiceCallError(status_code, detail)`.

1.10 Every service exposes

`GET /health` → `{"status":"ok","service":"<name>"}`, **no DB call** (Docker healthcheck uses it). `GET /ready` → checks DB + Redis + broker, 503 if any down. Swagger at `/docs`, `title="<Name> Service"`, `version="1.0.0"`.

1.11 Python service templates (build context is the REPO ROOT)

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONPATH=/app:/app/libs
COPY requirements-base.txt ./
COPY services/<NAME>/requirements.txt ./service-requirements.txt
RUN pip install --no-cache-dir -r service-requirements.txt
COPY libs /app/libs
COPY services/<NAME>/app /app/app
COPY services/<NAME>/alembic.ini /app/alembic.ini
COPY services/<NAME>/alembic /app/alembic
COPY services/<NAME>/entrypoint.sh /app/entrypoint.sh
RUN chmod +x /app/entrypoint.sh
EXPOSE <PORT>
CMD ["/app/entrypoint.sh"]
```

`services/<NAME>/requirements.txt` is just `-r ../../requirements-base.txt.entrypoint.sh` — **LF endings only** (`.gitattributes` enforces it):

```
#!/bin/sh
set -e
alembic upgrade head
exec uvicorn app.main:app --host 0.0.0.0 --port "${SERVICE_PORT}"
```

Alembic `env.py` must use the sync driver:

```
from common.config import settings
from common.db import Base, sync_url
from app.models import * # noqa — populate metadata
config.set_main_option("sqlalchemy.url", sync_url(settings.DATABASE_URL))
target_metadata = Base.metadata
```

Hand-write `versions/0001_initial.py`; do not rely on `--autogenerate`.

1.12 Frontend — the shell is already built, you only add pages

Already in the repo and **frozen**: `package.json` (Next 14.2.18, React 18.3.1, Tailwind 3.4.15, **lightweight-charts 4.2.0**), `tsconfig`, `tailwind.config.ts` (dark theme tokens

`bg/panel/panel2/line/ink/muted/up/down/accent/warn`), `app/layout.tsx`, `app/globals.css`, `lib/types.ts`, `lib/api.ts` (typed client for every route + 401 handling), `lib/auth.tsx` (`useAuth()`), `lib/ws.ts` (`useLivePrices()`, `subscribeMarket()`), `lib/format.ts` (money/price/qty/pct/validators), `components/ui.tsx` (`Card`, `Button`, `Input`, `Select`, `Badge`, `Tone`, `Table`, `Empty`, `Spinner`, `ErrorBox`), `components/Toast.tsx` (`useToast()`), `components/Protected.tsx`, `components/Navbar.tsx`.

Use these primitives. Do not invent new button/card styles — that is what keeps four people's pages looking like one product.

Page ownership (no two people touch the same file):

Route / component	Owner
app/page.tsx (dashboard), app/login, app/register, app/wallet, components/SymbolTable.tsx, components/DepositForm.tsx	B
app/orders/page.tsx, components/OrderTicket.tsx, OrderBook.tsx, RecentTrades.tsx, OrdersTable.tsx	C
app/market/[symbol]/page.tsx, app/portfolio/page.tsx, components/CandleChart.tsx, HoldingsTable.tsx	D
app/notifications/page.tsx, components/NotificationBell.tsx	A

`OrderTicket`, `OrderBook`, `RecentTrades` already exist as **compiling stubs** so D's market page builds from hour 0. C replaces the bodies; the file paths and props are frozen:

```
<OrderTicket symbol={string} lastPrice={string | null} onPlaced={() => void} />
<OrderBook   symbol={string} onPriceClick={(price: string) => void} />
<RecentTrades symbol={string} />
```

1.13 Merge protocol

- Branches: `feat/team-b-identity`, `feat/team-c-trading`, `feat/team-d-market-portfolio`. A works on `main`.
- **Nobody edits `libs/common`, the frontend shell, `docker-compose.yml`, `.env`, or another team's folder.** Need a change there? Message A.
- Push at least every 3 hours even if incomplete. A stale branch at hour 20 is how projects die.
- Each person delivers, in their own folders: `Dockerfile`, `requirements.txt`, `entrypoint.sh`, `alembic/`, `selfcheck.py`, and `SERVICE_NOTES.md` (endpoints, events handled, edge cases covered, deviations).
- Merge windows: **hour 13** and **hour 18**. `git merge --no-ff feat/...`

SECTION 2 — YOUR SERVICE 1: NOTIFICATION SERVICE (port 8007, `notification_db`)

Fire-and-forget alerts. Architecturally it exists to prove that a slow consumer **can never slow down the matching engine** — say exactly that in the README.

2.1 Layout

```
services/notification-service/
  app/{main.py,models.py,schemas.py,routes.py,handlers.py,emailer.py,deps.py}
  alembic/{env.py,versions/0001_initial.py}    alembic.ini
```

```
requirements.txt Dockerfile entrypoint.sh docker-compose.dev.yml
selfcheck.py SERVICE_NOTES.md
```

2.2 Model

```
class Notification(Base):
    __tablename__ = "notifications"
    id = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid.uuid4)
    user_id = mapped_column(UUID(as_uuid=True), nullable=False, index=True)
    type = mapped_column(String(32), nullable=False)
    title = mapped_column(String(120), nullable=False)
    message = mapped_column(String(500), nullable=False)
    symbol = mapped_column(String(10), nullable=True)
    reference_id = mapped_column(UUID(as_uuid=True), nullable=True) # order_id /
trade_id
    is_read = mapped_column(Boolean, nullable=False, default=False)
    created_at = mapped_column(DateTime(timezone=True),
server_default=func.now())
    __table_args__ = (Index("ix_notif_user_created", "user_id", "created_at"),)

class ProcessedEvent(Base):
    __tablename__ = "processed_events"
    event_id = mapped_column(UUID(as_uuid=True), primary_key=True)
    event_type = mapped_column(String(64))
    handled_at = mapped_column(DateTime(timezone=True), server_default=func.now())
```

Types: `ORDER_FILLED`, `ORDER_PARTIALLY_FILLED`, `ORDER_CANCELLED`, `ORDER_REJECTED`.

2.3 Endpoints

Method	Path	Notes
GET	<code>/notifications?</code> <code>unread_only=false&limit=20&offset=0</code>	JWT, newest first, <code>limit</code> capped at 200
GET	<code>/notifications/unread-count</code>	JWT → <code>{"count": n}</code> — powers the navbar badge
POST	<code>/notifications/{id}/read</code>	JWT, 404 if not yours (don't confirm it exists), idempotent
POST	<code>/notifications/read-all</code>	JWT → <code>{"marked": n}</code>

2.4 Event handlers

`trade.executed` → **two** notifications, one per side:

- buyer: `"Order filled" / "Bought 10 AAPL @ $195.50"`, type `ORDER_FILLED` if `buy_order_remaining == 0` else `ORDER_PARTIALLY_FILLED`

- seller: same shape with `Sold`, keyed off `sell_order_remaining`

`order.cancelled` → one notification.

- `USER_REQUEST` → "Order cancelled" / "Cancelled 7 unfilled AAPL shares"
- `NO_LIQUIDITY` → "Order could not be filled" / "No liquidity available for your market order"
- `IOC_REMAINDER` with `cancelled_quantity == 0` → **skip entirely**, nothing happened worth telling the user

`order.rejected` → "Order rejected" / the `reason` verbatim.

Every handler starts with `already_processed(redis, "notification", event_id)` and, on exception, calls `clear_processed(...)` before re-raising so the retry can re-run.

2.5 Mock email (the proposal asks for it)

`emailer.py` — one log line per notification:

```
EMAIL -> arham@x.com | Order filled | Bought 10 AAPL @ 195.50
```

Resolve the address from `GET http://user-service:8001/internal/users/{user_id}` with a tiny in-process cache (`dict`, 5-minute TTL, max 1000 entries). **If User Service is unreachable, log with the user id instead and carry on.** A notification must never fail because of an email lookup.

2.6 Edge cases — Notification Service

1. **Duplicate events** → Redis idem + `ProcessedEvent` PK, or the user gets the same alert five times after a broker retry.
2. **User Service down** → cached or degraded (`user_id` instead of email). Never raise.
3. **A market order sweeping 10 levels** creates 20 notifications. Acceptable; if you have spare time, coalesce per (`reference_id`, `type`) inside a 2-second window — but write first, tidy later. Never hold a message to coalesce.
4. **Message length** → truncate to 500 chars before insert; a long reject reason must not blow up the INSERT.
5. **Marking someone else's notification read** → 404, not 403.
6. **Unbounded growth** → index (`user_id`, `created_at DESC`), always paginate, cap `limit` at 200.
7. **Poison message** → `common.events.Broker` dead-letters after 5 attempts. Don't add your own retry loop.
8. **Money in message text** → `money_str` then 2 dp for humans: `f"${Decimal(price):.2f}"`.
9. **Self-trade** would give one user both notifications — the engine prevents it; be defensive anyway.
10. **read-all on 5000 rows** → a single `UPDATE ... WHERE user_id = :u AND is_read = false`, not a loop.

2.7 `selfcheck.py` (assert-based, no pytest)

```

insert via handler: trade.executed with buyer=U1, seller=U2, remaining 0/0
-> 2 rows, U1 title "Order filled", type ORDER_FILLED
same event again -> still 2 rows
partial fill (buy_order_remaining=5) -> buyer type ORDER_PARTIALLY_FILLED
order.cancelled reason=IOC_REMAINDER, cancelled_quantity=0 -> 0 rows
order.cancelled reason=USER_REQUEST, cancelled_quantity=7 -> 1 row
order.rejected -> 1 row, message contains the reason
unread-count for U1 == 2 ; read-all -> marked 2 ; unread-count == 0
mark someone else's notification -> 404

```

SECTION 3 — YOUR SERVICE 2: API GATEWAY (port 8000, no database)

```

services/gateway/
  app/{main.py, routing.py, proxy.py, ws_proxy.py, ratelimit.py, auth.py}
  requirements.txt  Dockerfile      # no alembic; entrypoint is just uvicorn

```

Routing. One ordered list of (prefix, upstream_base); longest prefix wins. Forwarded path = original minus /api.

Auth. Public prefixes: /api/auth/, /api/market/, /api/book/, /ws/market, /health, /docs, /openapi.json. Everything else needs a valid JWT (common.security.decode_token); forward Authorization unchanged plus X-User-Id for logging. **Any path containing /internal/ → 404.**

Rate limit.

```

key = f"ratelimit:{identity}:{int(time.time() // 60)}"
count = await redis.incr(key)
if count == 1: await redis.expire(key, 60)
if count > limit:
    raise HTTPException(429, "rate limit exceeded", headers={"Retry-After": "60"})

```

identity = user id when authenticated, else client IP. 120/min default, **30/min on POST /api/orders**. **If Redis is down, log a warning and allow the request** — rate limiting is not a correctness feature; don't take the exchange down for it. Exempt bot users (email ends @mse.local) or the market maker will 429 itself.

HTTP proxy. One shared httpx.AsyncClient created in the lifespan — never one per request, that leaks sockets:

```

client = httpx.AsyncClient(timeout=httpx.Timeout(15.0, connect=5.0),
follow_redirects=False)

```

Forward method, path, query, body and headers **minus hop-by-hop** (`host`, `connection`, `keep-alive`, `transfer-encoding`, `upgrade`, `content-length`). `httpx.ConnectError` → 503 `{"detail": "<service> unavailable"}`; `httpx.TimeoutException` → 504. **Return the upstream status and body untouched** — a 409 from Account must reach the frontend as a 409 with its `detail`, or nobody can debug anything.

WebSocket bridge for `/ws/market` — two tasks, first to finish cancels the other:

```
@app.websocket("/ws/market")
async def ws_market(ws: WebSocket):
    await ws.accept()
    qs = ws.scope.get("query_string", b "").decode()
    upstream = "ws://market-data-service:8005/ws/market" + (f"?{qs}" if qs else "")
    try:
        async with websockets.connect(upstream, ping_interval=20) as up:
            async def c2u():
                while True: await up.send(await ws.receive_text())
            async def u2c():
                async for msg in up: await ws.send_text(msg)
            done, pending = await asyncio.wait(
                {asyncio.create_task(c2u()), asyncio.create_task(u2c())},
                return_when=asyncio.FIRST_COMPLETED)
            for t in pending: t.cancel()
    except Exception:
        pass
    finally:
        with contextlib.suppress(Exception): await ws.close()
```

CORS. `allow_origins=["http://localhost:3000"]`, `allow_credentials=True`, all methods/headers.

3.1 Edge cases — Gateway

1. `/internal/` must be unreachable from outside — test it explicitly.
2. Upstream 4xx bodies must survive the proxy intact (status **and** `detail`).
3. Query strings must be preserved (`?limit=200&interval=5m`).
4. `content-length` must not be forwarded when the body is re-encoded → causes truncated responses.
5. WS bridge failure must close the client socket, not hang it.
6. Redis down → allow requests, log once per minute, never 500.
7. A slow upstream must 504 at 15 s, not hang the whole event loop.
8. `OPTIONS` preflight must return 200 without hitting an upstream.
9. Rate-limit key must include the minute bucket or it never resets.
10. Unknown path → 404 `{"detail": "no route"}`, not a stack trace.

SECTION 4 — PLATFORM (infrastructure you own)

4.1 `.env` and `.env.example`

```

COMPOSE_PROJECT_NAME=mse
JWT_SECRET=super-secret-change-me-in-production-0f3a9c
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
INTERNAL_API_KEY=internal-dev-key-7c1b
LOG_LEVEL=INFO
POSTGRES_USER=mse
POSTGRES_PASSWORD=mse_pw
REPLICATION_USER=replicator
REPLICATION_PASSWORD=replicator_pw
RABBITMQ_DEFAULT_USER=guest
RABBITMQ_DEFAULT_PASS=guest
RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672/
REDIS_URL=redis://redis:6379/0
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000

```

Commit both. A missing `.env` at 3 a.m. is a bigger risk than a leaked dev password — note that in the README.

4.2 `docker-compose.yml`

- 7 × `postgres:16-alpine`, own named volume each, `healthcheck: pg_isready -U mse`.
- `redis:7-alpine` (`redis-cli ping`), `rabbitmq:3.13-management-alpine` (`rabbitmq-diagnostics -q ping`, ports 5672 + 15672).
- 8 app services: `build: { context: ., dockerfile: services/<name>/Dockerfile }`.
- `depends_on` with `condition: service_healthy` everywhere.
- App healthcheck — **python:slim has no curl**:

```

test: ["CMD","python","-c","import urllib.request,sys; sys.exit(0) if
urllib.request.urlopen('http://localhost:8003/health').status==200 else
sys.exit(1)"]
interval: 10s   timeout: 5s   retries: 10   start_period: 30s

```

- One bridge network `mse-net` — **this is our Service Discovery** (Docker DNS resolves container names). Say so in the README.
- Env per service: `SERVICE_NAME`, `SERVICE_PORT`, `DATABASE_URL`, `REDIS_URL`, `RABBITMQ_URL`, `JWT_SECRET`, `JWT_ALGORITHM`, `JWT_EXPIRE_MINUTES`, `INTERNAL_API_KEY`, `LOG_LEVEL`, and the `*_SERVICE_URL` values.
- Market Data gets **both** `DATABASE_URL` (primary) and `DATABASE_REPLICA_URL` (replica).
- Frontend: `build: { context: ./frontend, args: { NEXT_PUBLIC_API_URL, NEXT_PUBLIC_WS_URL } }` — **NEXT_PUBLIC_* is inlined at build time; runtime environment: does nothing.**
- `restart: unless-stopped` everywhere.

```

postgresql+asyncpg://mse:mse_pw@postgres-user:5432/user_db
postgresql+asyncpg://mse:mse_pw@postgres-account:5432/account_db
postgresql+asyncpg://mse:mse_pw@postgres-order:5432/order_db
postgresql+asyncpg://mse:mse_pw@postgres-market-primary:5432/market_db
postgresql+asyncpg://mse:mse_pw@postgres-market-replica:5432/market_db
(DATABASE_REPLICA_URL)
postgresql+asyncpg://mse:mse_pw@postgres-portfolio:5432/portfolio_db
postgresql+asyncpg://mse:mse_pw@postgres-notification:5432/notification_db

```

4.3 PostgreSQL master-slave streaming replication (Market Data) — a graded requirement

`infra/postgres/primary/init-replication.sh` → mounted into `/docker-entrypoint-initdb.d/`:

```

#!/bin/bash
set -e
psql -v ON_ERROR_STOP=1 -U "$POSTGRES_USER" -d "$POSTGRES_DB" <<-EOSQL
    CREATE USER ${REPLICATION_USER} WITH REPLICATION LOGIN PASSWORD
    '${REPLICATION_PASSWORD}';
    SELECT pg_create_physical_replication_slot('market_replica_slot');
EOSQL
cat >> "$PGDATA/postgresql.conf" <<-EOF
    wal_level = replica
    max_wal_senders = 10
    max_replication_slots = 10
    hot_standby = on
    synchronous_commit = off
EOF
echo "host replication ${REPLICATION_USER} 0.0.0.0/0 md5" >> "$PGDATA/pg_hba.conf"

```

Replica container: **no POSTGRES_DB env** (it must not initdb). Override its command with

`infra/postgres/replica/setup-replica.sh`, run as user: `postgres`:

```

#!/bin/bash
set -e
if [ ! -s "$PGDATA/PG_VERSION" ]; then
    until pg_isready -h postgres-market-primary -U "$POSTGRES_USER"; do sleep 2;
done
rm -rf "${PGDATA:?}"/*
PGPASSWORD="${REPLICATION_PASSWORD}" pg_basebackup \
    -h postgres-market-primary -p 5432 -U "$REPLICATION_USER" \
    -D "$PGDATA" -Fp -Xs -P -R -S market_replica_slot
chmod 0700 "$PGDATA"
fi
exec docker-entrypoint.sh postgres

```

-R writes `standby.signal + primary_conninfo` for you (Postgres ≥ 12).

Verification (README + demo):

```
docker compose exec postgres-market-primary psql -U mse -d market_db -c "SELECT
client_addr,state,sync_state FROM pg_stat_replication;"
docker compose exec postgres-market-replica psql -U mse -d market_db -c "SELECT
pg_is_in_recovery();"

```

Expect one `streaming` row and `t`. **Insurance:** if `DATABASE_REPLICA_URL` is empty, Team D's service falls back to the primary and everything still works. Make sure the var exists.

4.4 Makefile (LF endings; also list raw commands in the README for Windows)

```
up:    docker compose up -d --build
down:  docker compose down
nuke:  docker compose down -v
logs:  docker compose logs -f --tail=100
seed:  python infra/seed/seed_market_data.py && python infra/seed/market_maker.py
mm:    python infra/seed/market_maker.py --loop
smoke: python scripts/smoke_test.py
repl:  docker compose exec postgres-market-primary psql -U mse -d market_db -c
"SELECT client_addr,state FROM pg_stat_replication;"

```

4.5 Edge cases — platform

1. **Startup ordering** — healthcheck-gated `depends_on` + `Broker.connect` retry (60×2 s) + `pool_pre_ping`. Never plain `depends_on`.
2. **CRLF** on `entrypoint.sh` → `exec /app/entrypoint.sh: no such file or directory`. `.gitattributes` is committed; verify with `file services/*/entrypoint.sh`.
3. **Build context** — every Dockerfile builds from repo root; `.dockerignore` is committed.
4. **JWT secret drift** — one wrong container 401s everything with no useful message. Have `/ready` log a SHA-256 prefix of the secret so you can eyeball mismatches.
5. **Replica must not initdb** — no `POSTGRES_DB` env, guard on `PG_VERSION`.
6. **Host port 5432 is often taken** by a local Postgres — that's why everything is 5433+.
7. **docker compose down without -v** keeps stale volumes with old schemas → `make nuke` after a model change.
8. **Docker Desktop memory** — 7 Postgres + RabbitMQ + Next.js needs ~8 GB. Set it *before* hour 13.
9. **Alembic race** — keep every service at 1 replica.
10. **NEXT_PUBLIC_* as runtime env does nothing** — must be build args.

SECTION 5 — FRONTEND: `/notifications` page +

bell

`components/NotificationBell.tsx` already exists as a working baseline (polls `/unread-count` every 5 s, dropdown of the latest 10, marks all read on open). **Upgrade it, keep the path** — `Navbar.tsx` imports it.

`app/notifications/page.tsx`: `<Protected>`, full paginated list, filter All / Unread, type icon and colour (`ORDER_FILLED` up, `ORDER_REJECTED` down, `ORDER_CANCELLED` muted), relative time via `timeAgo`, "Mark all read" button, `<Empty>` state. Use `Card`, `Table`, `Badge`, `Button` from `components/ui.tsx`.

SECTION 6 — DOCS (yours)

- `docs/architecture.md` — Mermaid system diagram (below).
- `docs/use-cases.md` — Mermaid version of the proposal's use-case diagram: Trader → Register/Login, View Live Market Data, Deposit Virtual Funds, Place Buy/Sell Order, Cancel Order, View Portfolio & P/L; «include» Validate Buying Power, Execute Order Matching, Receive Trade Notification.
- `docs/sequence-place-order.md` — the saga including the compensation branch.
- `docs/events.md` — the table from §1.7.
- `README.md` — one-command run, architecture summary, **a table mapping every proposal requirement to the file that implements it** (graders love this), replication verification, demo script, known limitations.

```
graph TB
    FE[Next.js Frontend :3000] -->|REST + WS| GW[API Gateway :8000]
    subgraph "JWT · rate limit · routing"
        GW --> US[User :8001] --> DBU[(user_db)]
        GW --> AS[Account :8002] --> DBA[(account_db)]
        GW --> OS[Order :8003] --> DBO[(order_db)]
        GW --> ME[Matching Engine :8004]
    end
    subgraph "in-memory order book"
        GW --> MD[Market Data :8005]
        GW --> PS[Portfolio :8006] --> DBP[(portfolio_db)]
        GW --> NS[Notification :8007] --> DBN[(notification_db)]
        MD -->|writes| MDP[(market_db PRIMARY)]
        MD -->|chart reads| MDR[(market_db REPLICA)]
        MDP -.→|streaming replication|. MDR
        OS -->|REST: reserve funds| AS
        OS -->|REST: reserve shares| PS
        OS -.→|order.accepted| MQ1[MQ{{RabbitMQ}}]
    end
    subgraph "exchange.events"
        MQ1 --> ME
        ME -.→|trade.executed / order.cancelled| MQ2[MQ]
        MQ2 --> OS & AS & PS & MD & NS
        AS <--> R[(Redis)]
    end
    subgraph "locks · cache · pub-sub"
        MD <--> R
        GW <--> R
    end
```

SECTION 7 — HOUR-BY-HOUR PLAN (24 h)

Hours	Work
0–1	Push <code>libs/common</code> , frontend shell, <code>requirements-base.txt</code> , the 4 team docs. Tell everyone to pull now.
1–4	<code>docker-compose.yml</code> with all 7 Postgres + Redis + RabbitMQ + placeholders for the 8 app services. Infra up, healthchecks green.
4–6	Postgres replication working — <code>pg_stat_replication</code> shows <code>streaming</code> .
6–8	Notification Service complete + <code>selfcheck.py</code> .
8–11	API Gateway complete: routing, auth, rate limit, HTTP proxy, WS bridge. Test against a throwaway echo service.
11–13	<code>/notifications</code> page + bell upgrade. <code>Makefile</code> .
13–14	Merge window 1. Pull B, C, D. Build all images. Fix only import/env/compose errors.
14–17	Integration debugging, money path first: register → deposit → order → fill → portfolio.
17–19	Merge window 2. Frontend pages in. Run B's <code>smoke_test.py</code> .
19–21	Fix what the smoke test finds. Run the market maker; watch charts and notifications live.
21–23	README + diagrams + requirement→code mapping. Screenshots: Swagger, RabbitMQ UI, replication query, charts, portfolio, a lock acquire/release log line.
23–24	Clean-clone test: <code>docker compose down -v && make up && make seed && make smoke</code> . Tag <code>v1.0.0</code> .

SECTION 8 — DEFINITION OF DONE

- ☐ `git clone && cp .env.example .env && make up` → 18 healthy containers.
- ☐ `make seed` populates charts, book and bot users; `make smoke all` ☒, exit 0.
- ☐ `make repl` shows a `streaming` replica and `pg_is_in_recovery() = t`.
- ☐ Notification Service: `selfcheck.py` passes; replaying an event creates no duplicate alert.
- ☐ Gateway: `/internal/` unreachable; 409 bodies pass through intact; 429 fires at 31 orders/min; WS bridge delivers ticks.
- ☐ Swagger reachable for all 7 services + gateway; RabbitMQ UI shows all 14 queues bound.
- ☐ README maps every proposal requirement to code.
- ☐ `localhost:3000`: register → deposit → chart → order → fill → portfolio → notification, live, no reload.